

补充

C++

```
1  #include<iostream>
2  using namespace std;
3  class Fraction{
4  private:
5      int num1;
6      int num2;
7      int gcd(int num1,int num2){
8          while(num2!=0)
9          {
10             int r=num1%num2;
11             num1=num2;
12             num2=r;
13         }
14         return num1; //辗转相除求最大公因数
15     }
16
17 public:
18     Fraction(int n1, int n2) {
19         num1 = n1;
20         num2 = n2;
21     }
22
23     Fraction() {
24         num1 = 0;
25         num2 = 0;
26     }
27     Fraction(int n1) {
28         num1 = n1;
29         num2 = 0;
30     }
31
32     void Show() {
33         cout <<num1 << "/" << num2<<endl;
34     }
35     friend void fswap(Fraction& a,Fraction& b){
36         int tn1=a.num1;
37         int tn2=a.num2;
38         a.num1 = b.num1;
39         a.num2 = b.num2;
40         b.num1 = tn1;
41         b.num2 = tn2;
42     }
43     Fraction operator-(const Fraction&b) {
44         Fraction result;
45         result.num1 = num1 * b.num2 - b.num1 * num2;
46         result.num2 = num2 * b.num2;
47         int divisor=gcd(result.num1,result.num2);
48         result.num1=result.num1/divisor;
49         result.num2=result.num2/divisor;
50         return result;
51     }
52 };
53
54 //StudybarCommentBegin
55 int main(int argc, char *argv[]) {
56     int x,y;
57     cin>>x>>y;
58     Fraction a(1,x),b(1,y),c(2);
59     fswap(a,b);
```

```

60     a.Show(); cout<<endl;
61     c.Show(); cout<<endl;
62     c=b-a;
63     c.Show();
64     return 1;
65 }
66 //StudybarCommentEnd

```

C++

```

1  #include <iostream>
2  #include <cmath>
3
4  using namespace std;
5
6  // 求两个数的最大公约数
7  int gcd(int a, int b) {
8      if (b == 0)
9          return a;
10     return gcd(b, a % b);
11 }
12
13 class Fraction {
14 private:
15     int numerator;    // 分子
16     int denominator; // 分母
17
18 public:
19     // 构造函数
20     Fraction(int num = 0, int denom = 1) {
21         numerator = num;
22         denominator = denom;
23         simplify(); // 简化分数
24     }
25
26     // 重载输出流运算符
27     friend ostream& operator<<(ostream& os, const Fraction& frac) {
28         if (frac.denominator == 1)
29             os << frac.numerator;
30         else
31             os << frac.numerator << "/" << frac.denominator;
32         return os;
33     }
34
35     // 重载输入流运算符
36     friend istream& operator>>(istream& is, Fraction& frac) {
37         char slash;
38         is >> frac.numerator >> slash >> frac.denominator;
39         frac.simplify();
40         return is;
41     }
42
43     // 重载比较运算符 ≠
44     friend bool operator≠(const Fraction& frac1, const Fraction& frac2) {
45         return (frac1.numerator * frac2.denominator ≠ frac2.numerator * frac1.denominator);
46     }
47
48     // 重载加法运算符 +
49     friend Fraction operator+(const Fraction& frac1, const Fraction& frac2) {
50         int num = frac1.numerator * frac2.denominator + frac2.numerator * frac1.denominator;
51         int denom = frac1.denominator * frac2.denominator;
52         return Fraction(num, denom);
53     }
54
55     // 简化分数
56     void simplify() {
57         int commonDivisor = gcd(abs(numerator), abs(denominator));
58         numerator /= commonDivisor;
59         denominator /= commonDivisor;
60         if (denominator < 0) {

```

```

60         numerator = -numerator;
61         denominator = -denominator;
62     }
63 }
64 };
65
66 int main(int argc, char *argv[]) {
67     Fraction a(1,2), b, c(2);
68     cout << a << "\n" << b << "\n" << c << '\n';
69     cin >> a >> b;
70     if (a  $\neq$  b) {
71         cout << " $\neq$ " << endl;
72     }
73     cout << a + b;
74     return 0;
75 }
76

```

C++

```

1  #include<iostream>
2  #include<iomanip>
3  #include<cmath>
4  #include<complex>
5  using namespace std;
6  template<typename T>
7  T myabs(T value)
8  {
9      return abs(value); //如果你将字符传递给 myabs 函数，
10     // 它会计算字符的绝对值，这是字符的 ASCII 或 Unicode 值的绝对值。
11 }
12
13 template<typename T>
14 double myabs(complex<T> z)
15 {
16     double x = z.real();
17     double y = z.imag(); // "imaginary part."
18     double r = sqrt(x * x + y * y); // 复数的绝对值（也称为模或大小）
19     // 是复平面上从原点到该复数所表示的点的距离。
20     return r;
21 }
22 //StudybarCommentBegin
23 int main()
24 { int a=-19;
25   char c=-1*'a';
26   double df=-123456.1234567;
27   float f=-123456.1234567;
28   complex<double> z(1.1,2.2); //myabs参数是复数时，求复数的模
29   cout<<fixed<<setprecision(8)<<myabs(a)<<"\n"<<myabs(df)<<"\n"<<myabs(f)<<"\n"<< myabs(c);
30   cout<<"\n"<<myabs(z)<<endl;
31   cin>>df>>f;
32   cout<<fixed<<setprecision(8)<<myabs(df)<<"\n"<<myabs(f)<<"\n";
33   if (df*100*100==df*10000) // 以及判断两个double是否相等要小心，通常不能简单= 或 $\neq$ 。
34   // 做差：给定精度
35   { cout<<"=";
36   }else
37   { cout<<" $\neq$ ";
38   }
39   return 0;
40 }
41 //StudybarCommentEnd

```

C++

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  class Clock24 {
6  private:

```

```

7     int hour, minute, second;
8 public:
9     bool IS24;
10    Clock24(int h = 0, int m = 0, int s = 0, bool is24 = true) : IS24(is24), hour(h % 24), minute(m % 60), second(s % 60) {}
11
12    int getSecondsOfDay() const {
13
14        return hour*3600 + minute*60 + second;
15    }
16
17    Clock24& operator++() {
18        ++second;
19        if (second == 60) {
20            second = 0;
21            ++minute;
22            if (minute == 60) {
23                minute = 0;
24                ++hour;
25                hour %= 24;
26            }
27        }
28        return *this;
29    }
30
31    Clock24 operator++(int) {
32        //它用于在当前对象的时间基础上增加一秒
33        //，并返回增加前的时间状态。
34        Clock24 temp(*this);
35        ++(*this); //对当前对象进行递增操作。
36        //由于前置递增操作符会修改当前对象的状态，
37        //因此在这里通过调用它来实现时间增加一秒的效果。
38        return temp;
39    }
40
41    friend istream& operator>>(istream& in, Clock24& c) {
42        in >> c.hour >> c.minute >> c.second;
43        return in;
44    }
45
46    friend ostream& operator<<(ostream& out, const Clock24& c) {
47        if (c.IS24)
48            out << setfill('0') << setw(2) << c.hour << ":" << setw(2) << c.minute << ":" << setw(2) << c.second
49            ;
50        else
51            out << setfill('0') << setw(2) << (c.hour % 12 == 0 ? 12 : c.hour % 12) << ":" << setw(2) << c.minute << ":" << setw(2) << c.second << (c.hour ≥ 12 ? " PM" : " AM");
52        return out;
53    }
54 }
55
56 //StudybarCommentBegin
57 int main()
58 {
59     const Clock24 t0(0,1,2,false); //第四个参数若为false,表示用12小时制显示时间。
60     cout<<"t0 is "<< t0 <<" (注意0点的12小时制为 12:XX:XX AM)";
61     cout<<"\nt0是一天中的"<<t0.getSecondsOfDay()<<"秒";
62     Clock24 t1(12,1,2,false), t2(t0), t3(23,45,2);
63     cout<<"\nt1 is "<<t1<<" (12点的12小时制显示为 12:XX:XX PM) ";
64     t2.IS24=true; //用于设置是24小时制(true),还是12小时制(false)的显示。
65     cout<<"\nt2 is "<<t2<<"\nt3 is "<<t3<<endl;
66     t3.IS24=false;
67     cout<<"(++t3)++ is "<<(++t3)++<<endl;
68     cout<<"t3 is "<<t3<<endl;
69
70     cin >> t2;
71     cout<<"(++(++t2))++ is "<<(++(++t2))++<<endl; //后置递增操作符(++)的重载
72     t2.IS24=false;
73     cout<<"t2的12小时制: "<<t2;
74     cout<<"\nt2是一天中的"<<t2.getSecondsOfDay()<<"秒"<<endl;

```

```
73
74     return 0;
75 }
76 //StudybarCommentEnd
77
```

动态

C++

```
1  #include <iostream>
2  using namespace std;
3  class Point {
4  private:
5      int x;
6      int y; //x,y数据成员
7  public:
8      Point() : x(0), y(0) {
9          cout<<"Point(int,int) is called."<<endl;
10     }
11     Point(int x, int y) : x(x), y(y) {
12         cout<<"Point(int,int) is called."<<endl;
13     }
14     Point(const Point &p) : x(p.x), y(p.y) { // 本类对象初始化新对象
15         // 创建了一个对象b, 并将对象a作为参数传递给拷贝构造函数。
16         // 这里的拷贝构造函数会创建一个新的对象, 其内容和a相同。
17         // 这个过程被称为拷贝初始化。
18         cout<<"Point(const Point &) is called."<<endl;
19     }
20
21     Point& operator=(const Point &p) {
22         // 这一行将对象a的值赋给对象c。这里并不是创建一个新的对象,
23         // 而是将对象a的值复制给对象c。因此, 赋值运算符重载函数会被调用。
24         // 在这个例子中, 对象c已经存在,
25         // 赋值运算符重载函数会将对象a的值复制给对象c, 覆盖原有的值。
26         x = p.x;
27         y = p.y;
28         cout<<"operator=(const Point &p) is called."<<endl;
29         return *this;
30     }
31     ~Point() {
32         cout<<"~Point() is called."<<endl;
33         // 当对象超出作用域、被删除或程序结束时, 编译器会自动调用析构函数。
34         // 在这个程序中, 每个对象在超出作用域时都会调用析构函数, 以确认对象被正确销毁。
35     }
36 };
37 //StudybarCommentBegin
38 int main()
39 {
40     Point a(1,2);
41     Point b(a);
42     Point c;
43     c=a;
44     return 0;
45 }
46 //StudybarCommentEnd
```

C++

```
1  #include <iostream>
2  #include <cstdlib> // 用于调用exit函数
3  using namespace std;
4
5  class VectorDouble {
6  private:
7      double* array; // 指向double类型的指针
8      unsigned int capacity_; // 动态数组的容量
9      unsigned int size_; // 动态数组中实际存储元素的个数
10
11  public:
12      // 默认构造函数
```

```

13     VectorDouble() {
14         capacity_ = 0;
15         size_ = 0;
16         array = NULL;
17     }
18
19     // 带有两个参数的构造函数
20     VectorDouble(unsigned int cap, double init_value = 0) {
21         capacity_ = cap;
22         size_ = cap;
23         array = new double[capacity_];
24         for (unsigned int i = 0; i < capacity_; ++i) {
25             array[i] = init_value;
26         }
27     }
28
29     // 析构函数
30     ~VectorDouble() {
31         delete[] array;
32     }
33
34     // 返回数组容量
35     unsigned int capacity() const {
36         return capacity_;
37     }
38
39     // 返回数组实际存储元素的个数
40     unsigned int size() const {
41         return size_;
42     }
43
44     // 重载下标运算符
45     double& operator[](unsigned int i) {
46         if (i ≥ size_) {
47             cout << "Out of Range!" << endl;
48             exit(EXIT_FAILURE);
49         }
50         return array[i];
51     }
52 };
53
54 int main() {
55     VectorDouble v1;
56     cout << v1.capacity() << endl;
57     cout << v1.size() << endl;
58
59     double value;
60     cin >> value;
61     VectorDouble v2(5, value);
62     cout << v2[0] << endl;
63     cout << v2[4] << endl;
64
65     int capacity;
66     cin >> capacity;
67     VectorDouble v3(capacity);
68     cout << v3.capacity() << endl;
69     cout << v3.size() << endl;
70     for (int i = 0; i < capacity; i++) {
71         v3[i] = i * 1.1;
72     }
73     cout << v3[5] << endl;
74     cout << v3[31] << endl;
75
76     return 0;
77 }

```